

Lab07 Kaleo256333

Kaleo Campos

2026-09-23

```
library(DBI)
library(RSQLite)
```

Questão 1

Baixe o arquivo `uwmadison.sqlite3`. Conecte-se a ele usando o pacote `RSQLite`, armazenando a conexão em uma variável `conn`.

```
conn <- dbConnect(SQLite(), "uwmadison.sqlite3")

dbListTables(conn)
```

```
[1] "course_offerings"      "courses"              "grade_distributions"
[4] "instructors"           "rooms"                 "schedules"
[7] "sections"              "subject_memberships"   "subjects"
[10] "teachings"
```

Questão 2

Quem são e quantos são os professores que lecionaram disciplinas cujo tópico era estatística (`subjects.abbreviation='STAT'`)?

```
ofertas_stat <- paste(
  "SELECT course_offering_uuid FROM subject_memberships",
  "WHERE subject_code IN",
  "(SELECT code FROM subjects WHERE abbreviation = 'STAT')")
)
```

```

sql <- paste(
  "SELECT DISTINCT i.id AS id, i.name AS professor",
  "FROM instructors i",
  "INNER JOIN teachings t ON i.id = t.instructor_id",
  "INNER JOIN sections sec ON t.section_uuid = sec.uuid",
  "WHERE sec.course_offering_uuid IN (", ofertas_stat, ")",
  "ORDER BY professor"
)

professores_stat <- dbGetQuery(conn, sql)
nrow(professores_stat)

```

[1] 379

São 379 professores. A lista completa, em ordem alfabética:

```
knitr::kable(professores_stat, row.names = FALSE)
```

id	professor
4787911	ABIGAIL BENZINE
3111842	ADIN-CRISTIAN ANDREI
3775809	AKICHIKA OZEKI
4958796	ALAN HUANG
5451706	ALBERTO DEL PIA
3661782	ALBRECHT KLEMM
5656034	ALEXANDER COVINGTON
4680082	ALEXANDER FISH
3573775	ALEXANDER KISELEV
5162339	ALYSSA DIGILIO
129415	ANATOLE BECK
3674168	ANDREJ ZLATOS
5003565	ANDREW LESLIE
4952769	ANNE BRUCKNER
5498092	ANQI SHI
4646729	ANRU ZHANG
2600893	ARNOLD MILLER
4963378	AUGUST JENSEN
5852265	BEHZAD AALIPUR
5694563	BEHZAD AALIPUR HAFSHEJANI
4140469	BEN ADAM HAALAND

id	professor
4538519	BENEDEK VALKO
4632211	BENJAMIN RECHT
5906523	BI CHENG WU
4155990	BIN DAI
5186047	BIN ZHANG
4436066	BIN ZHU
5144884	BINGYING XIE
3484236	BO HUANG
5632570	BO YANG
5690247	BOWEN HU
4753018	BRET HANLON
751075	BRET LARGET
2602055	BRIAN YANDELL
5011312	BROOK LUERS
3955546	CECILE ANE
5883206	CHAN PARK
3976099	CHANHAN HSU
4724058	CHAOQUN MEI
4712098	CHAOYANG YU
3376532	CHELSEY GREEN
4586558	CHEN CHENG
5018463	CHEN JING
5592111	CHENGNING ZHANG
4597046	CHENLIANG XU
4903489	CHENSHENG KUANG
3973813	CHENXI LI
3629214	CHIA-CHIEH LIN
3613299	CHIEN-WEI CHEN
340544	CHRISTINA M. KENDZIORSKI
138086	CHRISTINE SORKNESS
3855025	CHRISTOPHER WAGNER
5887049	CLAIRE BOBST
4721578	CLAUDIA SOLIS LEMUS
4312759	COLE COOK
5332940	COLIN LONGHURST
5498648	CRYSTAL CHEN
5015194	CUICUI QI
5176022	CUIZE HAN
5846795	DANIELE CAPPELLETTI
4308515	DAVID ANDERSON
2601596	DAVID DEMETS

id	professor
2602109	DAVID GRIFFEATH
6043299	DEBRAJ DAS
4264265	DEREK BEAN
5305675	DEREK NORTON
3773167	DEYUAN JIANG
2600233	DIETRICH UHLENBROCK
393380	DONALD PORTER
4898014	DONG XIA
4897725	DONGGYU KIM
5620719	DONGHYUN LEE
2602050	DOUGLAS M. BATES
4728844	DUY NGUYEN
5516726	DUZHE WANG
5189535	EDWARD ERKER
2824494	ELOISA D CHAVAS
3566378	ERICA LEE DEADMAN
2602042	ERIK NORDHEIM
5204089	FAN GAO
4437355	FAN YANG
3460189	FANG FANG
3783301	FANGFANG WANG
4601948	FLORIAN BERTRAND
361804	FREDERICK BOEHM
5214164	GARVESH RASKUTTI
495087	GARY HOWARD SCHROEDER
4957688	GENG LI
5017655	GINA BENNINGER
5862942	GINA OH
5320625	GONZALO CONTADOR
2600488	GRACE WAHBA
4761200	GREGORIO MORENO-FLORES
5127444	GREGORY SHINAULT
4272783	GUANHUA CHEN
4436002	GUANNAN SUN
1505756	GUILHERME ROSA
4886028	GUILHERME VIEIRA NUNES LUDWIG
4884161	GUN WOONG PARK
5010171	HAN CHEN
5514514	HAO CHEN
5520701	HAO TENG
4435790	HAO ZHENG

id	professor
5323690	HAO ZHOU
3456935	HAODA FU
4725072	HAOYANG FAN
4149843	HEATHER MARIE BRAZEAU
3623432	HUAIBAO FENG
3483534	HUI WANG
5168069	HUIKUN ZHANG
5167990	HUILIN HU
5321537	HYEBIN SONG
4734607	HYUNSEUNG KANG
801445	IAN BRANSTAD RILEY
143932	ISMOR FISCHER
5515948	JAMES ANDERSON
2601633	JAMES D KUELBS
4636315	JARED BROWN
3011936	JASON P FINE
3956063	JASON RICHARD SWANSON
4140397	JEE YEON KIM
4890283	JEEA CHOI
4636145	JENNIFER BIRSTLER
4613728	JENNIFER NGUYEN
3627934	JESSE THOMAS HOLZER
4590715	JIAJIE CHEN
4433317	JIALE XU
4603165	JIAN WU
4886987	JIANCHANG HU
5410648	JIE SONG
4292460	JIE WEI
4436368	JIE ZHANG
4734274	JILI WANG
3783649	JINGCI MENG
3973318	JINGJIANG PENG
4902482	JINGLAN LI
4286103	JIWEI ZHAO
4880968	JOHN DAVIS
608997	JOHN GILLET
3806328	JOHN KANE
4896511	JOHN WILTSHIRE-GORDON
1634418	JONATHON PETERSON
1126199	JOSEP GINEBRA
4610245	JOSEPH DEUTSCH

id	professor
4731731	JU HEE CHO
3981051	JUN LI
1079018	JUN SHAO
3620114	JUN YIN
3781764	JUN ZHANG
321374	JUN ZHU
3095399	JUNGWON MUN
3620552	JUNHEE HAN
4541633	JUNHO LEE
5012817	KAILEI CHEN
2602032	KAM-WAH TSUI
4541610	KARL BROMAN
4929500	KARL ROHE
4491860	KATHERINE GOODE
3777992	KAZUHIKO SHINKI
4261114	KEEGAN KORTHAUER
3127329	KEN ONO
3978606	KEVIN HASEGAWA ENG
4744350	KEVIN PACKARD
3404299	KJELL DOKSUM
5476019	KRISHNAKUMAR BALASUBRAMANIAN
4199106	KRISTEN CYFFKA
5697552	KUNLING HUANG
5317373	KYLE HEBERT
5478075	KYLE HERBET
207204	KYUNGSMANN KIM
4435446	LAM HO
5521838	LAN LUO
3103459	LANCINE KONATE
3443667	LEI XU
3437432	LEV BORISOV
4845798	LIAM JOHNSTON
4277002	LIE XIONG
3980675	LILI LAN
5693114	LILI ZHENG
4578330	LILUN DU
5514932	LIN QI
5015113	LONG PHAN
4726678	LU MAO
5010560	LU YANG
5017774	LUWAN ZHANG

id	professor
5192960	LUXI CAO
5302154	MANJUSHA KANCHARLA
2601674	MARI PALTA
4195000	MARIA KAMENETSKY
5167906	MARIA ROJO
245914	MARIAN R FISHER
333334	MARY LINDSTROM
3467530	MATTHEW BALLARD
4280220	MENG SONG
1114818	MICHAEL FERRIS
783769	MICHAEL GEORGE ILTIS
270258	MICHAEL HOGAN
5708167	MICHAEL KUTZLER
5840930	MICHAEL LIOU
2601916	MICHAEL NEWTON
989414	MICHAEL RENE KOSOROK
394810	MICHELLE HARRIS
6029366	MIHAELA IFRIM
3575433	MIN JUNG LEE
3965653	MIN NIU
4133622	MING XIE
3077702	MING YUAN
4392906	MINJING TAO
4947463	MITCHELL PAUKNER
3404298	MOO K CHUNG
5514561	MUHONG GAO
2601621	MURRAY CLAYTON
1505727	NATALIA DE LEON GATTI
232828	NELLIE LAUGHLIN
3523865	NICHOLAS HENDERSON
640407	NICHOLAS STEPHEN KEULER
5523214	NING FAN
3882760	NORBERT BINKIEWICZ
2600689	NORMAN DRAPER
2601093	PAUL M TERWILLIGER
967335	PAUL RATHOUZ
5631398	PAUL SAVARIAPPAN
5326263	PEIGEN ZHOU
4149787	PERLA REYES
1515839	PHILIP WOOD
4633355	QI JIANG

id	professor
3974100	QI TANG
3338237	QIAN ZHIGUANG
5718599	QING LI
4460098	QIONG ZHANG
4582107	QIURONG CUI
4432994	QUEFENG LI
4161984	QUOC TRAN
789363	REBECCA KOSCIK
5488796	REBECCA POST
2600141	RICHARD A BRUALDI
2601613	RICHARD A. JOHNSON
2601926	RICHARD J. CHAPPELL
4957895	ROBERT HARRON
2601512	ROBERT R MEYER
2601091	ROBERT WARDROP
3780010	ROBERT WAYNE GREEN
893827	RONALD GANGNON
5887858	RONGJUN ZHU
5690182	RUI CHEN
4128750	RUI TANG
3987897	RUIFANG SONG
3343055	RUIFENG XU
5898424	RUNGANG HAN
5020061	RUOSI GUO
3727371	RYAN ZEA
4752238	SAMUEL STECHMANN
3981781	SANGBUM CHOI
5354849	SCOTT HOTTOVY
3253489	SCOTT JOSEPH HETZEL
5068296	SEAN KENT
5110390	SEBASTIEN ROCH
4733192	SEHO PARK
5520811	SEULKEE YUN
3573852	SEUNGBONG HAN
5905014	SHAN LU
982957	SHANE HUBLER
5515201	SHANG WU
4146030	SHENG WANG
5906594	SHENG WANG
4285558	SHENG ZHANG
5017571	SHENGJI JIA

id	professor
3127330	SHI JIN
4885477	SHIXUE LIU
4598772	SHIZHEN WANG
4699427	SHUAI CHEN
4271924	SHUANG HUANG
5170885	SHULEI WANG
4588354	SHUYUN YE
3622809	SIJIAN WANG
5323628	SIJING LI
5176905	SOHEIL SADEGHI
4593965	SOKOL VAKO
3955531	SONA Z SWANSON
4953246	SONG WANG
3884290	SRIKANTHMADHAVAN ARAVAMUTHAN
2600973	STEFFEN LEMPP
3122658	STEPHEN AARON STANHOPE
4538540	STEPHEN BERG
2601051	STEPHEN WAINGER
3382451	STEPHEN WRIGHT
5452538	STEVEN SAM
3117835	SUKHENDU MEHROTRA
3801422	SUNDUZ KELES
4590738	TAERI UHM
3781422	TAO YU
5023414	THEVAASIINEN CHANDERENG
417071	THOMAS COOK
2601039	THOMAS G. KURTZ
4904255	THU LE
4896939	TIEN VO
3374880	TIMO SEPPALAINEN
4879129	TIMOTHY IDOWU
4957916	TIMOTHY IDOWU
5520995	TING YE
3351080	TING-LI LIN
5329574	TONG LI
3230527	TONGHAI YANG
4896172	TRAM TA
5567363	TUN LEE NG
5664494	TZU HSIANG HUNG
3959251	VARSHA KULKARNI
4641200	VICTOR LUO

id	professor
4788841	VICTORIA MANSFIELD
4434130	WAI TONG FAN
3780817	WEI ZHENG
2602108	WEI-YIN LOH
4590481	WENWEN ZHANG
5322094	WENZHI CAO
5161294	WESLEY CHANG
4591139	XIAO GUO
4905505	XIAO NIE
3343005	XIAODAN WEI
5170801	XIAOMAO LI
4729481	XIAOPING FENG
5258581	XIAOWU DAI
4285528	XIN LI
4588103	XIN ZHANG
4726666	XINJIE HE
3620612	XINWEI DENG
4286679	XINXIN YU
4430246	XINYU SONG
4145889	XIPEI YANG
5170924	XIRAN WANG
4194670	XIUFENG SHAO
5322566	XIUYU MA
3791802	XIWEN MA
4286138	XU HE
4590559	XU XU
4726703	XUEYAO CHEN
3668575	XUN ZHAO
4426932	YAJUAN SI
3800708	YALI WANG
4721620	YAN CHEN
3457484	YANBING ZHENG
4285354	YANG ZHAO
4172260	YANNAN QIU
4895244	YAOGUO XIE
4576059	YAoyao XU
4695492	YAZHEN WANG
4285538	YI CHAI
5327139	YI LI
4586128	YI LIU
5693277	YIFAN MEI

id	professor
5323691	YILIN ZHANG
5170492	YING ZHANG
5696436	YONGFENG WU
5644228	YONGJOON KIM
5518378	YONGSU LEE
5514517	YOUNG LEE
5177493	YOUNG MIN PARK
4278664	YOUNGDEOK HWANG
5326059	YOURAN QI
3010828	YU MENGANG
3961448	YU QIU LIU
3791787	YUAN JIANG
5167973	YUAN LI
4733966	YUAN WANG
4906458	YUANZHI LI
5903922	YUCHANG WU
5700058	YUCHEN ZHOU
4144479	YUJIN CHUNG
5907599	YUNONG LIN
5214858	YUQING XU
1426304	ZHANG CHUNMING
4108525	ZHENGJUN ZHANG
3107260	ZHENGXIAO WU
4588525	ZHIGENG GENG
3339239	ZHIGUO XIAO
5693629	ZHONGJIE YU
3626007	ZHUANG WU
5099221	ZIFENG ZHAO
5591917	ZIJIAN NI
4138542	ZUOFENG SHANG

Questão 3

O GPA americano é definido numa escala de 0 a 4, em que $A = 4$, $AB = 3.5$, $B = 3$, $BC = 2.5$, $C = 2$, $D = 1$ e $F = 0$. Determinando a nota média de cada oferecimento pela ponderação da quantidade de alunos em cada extrato com os valores numéricos de cada conceito, indique (no que se refere a disciplinas no assunto de estatística):

- Quem é o professor mais difícil?
- Quem é o professor mais fácil?

- Qual é a disciplina mais difícil?
- Qual é a disciplina mais fácil?

```
notas <- paste(
  "SELECT gd.course_offering_uuid AS oferecimento,",
  "gd.section_number AS secao,",
  "4.0*gd.a_count + 3.5*gd.ab_count + 3.0*gd.b_count +",
  "2.5*gd.bc_count + 2.0*gd.c_count + 1.0*gd.d_count AS pontos,",
  "gd.a_count + gd.ab_count + gd.b_count + gd.bc_count +",
  "gd.c_count + gd.d_count + gd.f_count AS alunos",
  "FROM grade_distributions gd",
  "WHERE gd.course_offering_uuid IN (", ofertas_stat, ")"
)
```

Para comparar professores e disciplinas, agregamos as notas válidas ponderando pelo número de alunos; em caso de empate, apresentamos todos os resultados.

Nota média por oferecimento

```
sql <- paste(
  "SELECT g.oferecimento, c.name AS disciplina,",
  "SUM(g.pontos) / SUM(g.alunos) AS gpa_medio,",
  "SUM(g.alunos) AS alunos",
  "FROM (", notas, ") g",
  "INNER JOIN course_offerings co ON g.oferecimento = co.uuid",
  "INNER JOIN courses c ON co.course_uuid = c.uuid",
  "GROUP BY g.oferecimento, c.uuid, c.name",
  "HAVING SUM(g.alunos) > 0",
  "ORDER BY gpa_medio, g.oferecimento"
)

gpa_oferecimentos <- dbGetQuery(conn, sql)
nrow(gpa_oferecimentos)
```

```
[1] 663
```

```
knitr::kable(head(gpa_oferecimentos[c("disciplina", "gpa_medio", "alunos")], 10),
              digits = 2)
```

disciplina	gpa_medio	alunos
Introduction to Stochastic Processes	2.44	34
Introduction to the Theory of Probability	2.61	65
Linear Programming Methods	2.62	55
Introduction to Probability and Mathematical Statistics I	2.63	71
Linear Programming Methods	2.64	49
Introduction to Probability and Mathematical Statistics II	2.66	69
Introduction to Theory and Methods of Mathematical Statistics II	2.68	61
Introduction to Probability and Mathematical Statistics I	2.68	68
Introduction to the Theory of Probability	2.69	152
Linear Programming Methods	2.70	57

Professores

```
professores_secoes <- paste(
  "SELECT DISTINCT t.instructor_id,",
  "sec.course_offering_uuid AS oferecimento, sec.number AS secao",
  "FROM teachings t",
  "INNER JOIN sections sec ON t.section_uuid = sec.uuid"
)

sql <- paste(
  "SELECT i.id AS id, i.name AS professor,",
  "SUM(g.pontos) / SUM(g.alunos) AS gpa_medio,",
  "SUM(g.alunos) AS alunos",
  "FROM (", notas, ") g",
  "INNER JOIN (", professores_secoes, ") ps",
  "ON g.oferecimento = ps.oferecimento AND g.secao = ps.secao",
  "INNER JOIN instructors i ON ps.instructor_id = i.id",
  "GROUP BY i.id, i.name",
  "HAVING SUM(g.alunos) > 0",
  "ORDER BY gpa_medio, professor, id"
)

gpa_professores <- dbGetQuery(conn, sql)
```

Professor mais difícil

```
professores_dificeis <- gpa_professores[
  gpa_professores$gpa_medio == min(gpa_professores$gpa_medio), ]

knitr::kable(professores_dificeis, digits = 2, row.names = FALSE)
```

id	professor	gpa_medio	alunos
2601633	JAMES D KUELBS	2.64	40

Professores mais fáceis

```
professores_faceis <- gpa_professores[
  gpa_professores$gpa_medio == max(gpa_professores$gpa_medio), ]

nrow(professores_faceis)
```

[1] 3

```
knitr::kable(professores_faceis, digits = 2, row.names = FALSE)
```

id	professor	gpa_medio	alunos
4272783	GUANHUA CHEN	4	16
3884290	SRIKANTHMADHAVAN ARAVAMUTHAN	4	16
4426932	YAJUAN SI	4	11

Disciplinas

```
sql <- paste(
  "SELECT c.uuid AS id, c.name AS disciplina,",
  "SUM(g.pontos) / SUM(g.alunos) AS gpa_medio,",
  "SUM(g.alunos) AS alunos",
  "FROM (", notas, ") g",
  "INNER JOIN course_offerings co ON g.oferecimento = co.uuid",
  "INNER JOIN courses c ON co.course_uuid = c.uuid",
  "GROUP BY c.uuid, c.name",
  "HAVING SUM(g.alunos) > 0",
```

```

"ORDER BY gpa_medio, disciplina, id"
)

gpa_disciplinas <- dbGetQuery(conn, sql)

```

Disciplina mais difícil

```

disciplinas_dificeis <- gpa_disciplinas[
  gpa_disciplinas$gpa_medio == min(gpa_disciplinas$gpa_medio), ]

knitr::kable(disciplinas_dificeis[c("disciplina", "gpa_medio", "alunos")],
  digits = 2, row.names = FALSE)

```

disciplina	gpa_medio	alunos
Introduction to the Theory of Probability	2.89	2231

Disciplinas mais fáceis

```

disciplinas_faceis <- gpa_disciplinas[
  gpa_disciplinas$gpa_medio == max(gpa_disciplinas$gpa_medio), ]

nrow(disciplinas_faceis)

```

[1] 4

```

knitr::kable(disciplinas_faceis[c("disciplina", "gpa_medio", "alunos")],
  digits = 2, row.names = FALSE)

```

disciplina	gpa_medio	alunos
Data Science Practicum	4	12
Empir Proc&Semiparmtrc Infernc	4	8
Nonparametric Statistics and Machine Learning Methods	4	24
Sample Survey Theory and Method	4	54

Questão 4

Desconecte do banco de dados.

```
dbDisconnect(conn)
```

```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-09-23 20:04:27 -03"
```